

Lemon Aid 1주차 프로젝트 브리핑

기업 과제 수행 방향과 서비스화 관점을 파트별 산출물 중심으로 정리한 발표용 통합 문서입니다.

Lemon Aid는 음식·영양제 입력을 기반으로 만성질환자의 건강관리 흐름을 돕는 AI 기반 건강관리 보조 서비스입니다.

이 문서는 1주차 개별 문서를 모두 설명하는 대신, 발표에서 꼭 전달해야 할 핵심만 파트별로 압축한 통합본입니다.

1주차에는 최종 기능을 완성하기보다, 각 파트가 앞으로 어떤 기준으로 구현을 이어갈지 정리했습니다. UX는 사용자가 결과를 받아들이는 화면 규칙을 정했고, 음식 비전은 음식 입력을 분석 후보로 바꾸는 파이프라인을 잡았습니다. 알고리즘은 공식 기준과 운영 규칙을 분리했고, DB/백엔드는 사용자 기록과 Agent 분석 결과를 안전하게 연결하는 구조를 정리했습니다.

파트	1주차 핵심 산출물
UX	7개 화면 뼈대, 사용자 멈춤 지점, 결과 카드 공통 규칙
음식 비전	음식 후보 생성, 100g 기준 영양 프로필, 사용자 입력량 기반 재계산
알고리즘	공식 계산식과 운영 규칙 분리, preview 처리, 안전 표현 원칙
DB/백엔드	user_id 기준 데이터 연결, PostgreSQL/Redis 구조, Agent Memory

2.1 1주차 UX 작업의 핵심

UX 파트의 1주차 작업은 최종 화면 디자인이 아니라, **사용자가 AI·DB·음식 비전 결과를 이해하고 고칠 수 있게 만드는 화면 규칙**을 정한 것입니다.

서비스의 기술 결과가 아무리 좋아도 사용자가 어디서 시작해야 하는지 모르거나, AI 결과를 믿지 못하거나, 틀린 결과를 고치기 어렵다면 서비스 가치는 떨어집니다. 그래서 UX는 화면을 예쁘게 꾸미기 전에 사용자가 실제로 멈추는 지점을 먼저 정리했습니다.

2.2 7개 핵심 화면

1주차에는 서비스 전체 흐름을 구성하는 7개 기본 화면의 뼈대를 정했습니다.

코드	화면	역할
S-01	Splash	앱 시작 화면
S-02	Login	기존 사용자 로그인
S-03	Signup	신규 사용자 가입
S-04	Verify	이메일 확인 또는 인증
S-05	Consent	개인정보·건강정보·권한 안내
S-06	Onboarding	첫 사용법과 기본 정보 입력
S-07	Dashboard	기록, 분석 결과, 다음 행동을 보여주는 홈

이 화면들은 최종 디자인이 아니라, 다음 단계에서 AI·DB·음식 비전 결과가 들어올 자리를 미리 정해 둔 구조입니다.

2.3 사용자 기준

Lemon Aid의 사용자는 단순히 건강 앱을 둘러보는 사람이 아니라, 식사 직후나 약통 옆에서 짧은 시간 안에 기록을 남겨야 하는 사람입니다.

기준	UX 해석
주 사용자	30~50대 만성질환 관심층, 60대 이상도 함께 고려
사용 시간	한 번 켜 때 30초~2분, 하루 3~5회
사용 상황	식탁 앞, 약통 옆, 운동 직후
화면 원칙	큰 글씨, 큰 버튼, 높은 색 대비, 빠른 카메라 접근

따라서 UX는 긴 설명보다 짧은 안내, 복잡한 메뉴보다 바로 실행할 수 있는 카메라와 결과 카드, 작은 버튼보다 손가락 크기의 조작 영역을 우선합니다.

2.4 사용자가 멈추는 4가지 지점

지점	사용자 생각	UX가 해결할 일
처음 켤 때	앱을 켜는데 뭐부터 하지?	안내를 3단계 안으로 줄이고 카메라까지 빠르게 이동
결과를 의심할 때	AI 결과를 믿어도 되나?	확신도, 출처, 고치기 경로를 함께 표시
고치고 싶을 때	틀린 것 같은데 어떻게 고치지?	다른 화면으로 보내지 않고 같은 카드 안에서 수정
하루 마무리	오늘 잘했나? 내일 뭐 해야 하지?	점수와 함께 다음 할 일 한 줄 제공

이 지점들은 UX 혼자 해결할 수 없습니다. AI 팀은 확신도와 출처를 내려줘야 하고, DB 팀은 빈 상태와 사용자 데이터를 구분해줘야 하며, 음식 비전 팀은 후보와 검토 필요 상태를 화면이 이해할 수 있는 형식으로 제공해야 합니다.

2.5 화면 판단 기준과 결과 카드 약속

UX 파트는 화면을 정할 때 다음 6가지 기준을 사용합니다.

기준	내용
한눈에 중요한 것	한 화면에서 가장 중요한 것 하나를 먼저 보이게 함
손가락이 닿는 거리	자주 쓰는 일은 세 번 안에, 카메라는 한 번에 도달
신뢰 표시	AI 결과에는 출처, 확신도, 시간 중 최소 두 가지 표시
가독성	글자는 작지 않게, 버튼은 충분히 크게, 색 대비는 명확하게
같은 의미는 같은 모양	같은 행동은 같은 위치·색·아이콘으로 통일
빠져나갈 길	오류, 데이터 없음, 로딩에는 다음 행동 한 줄 제공

모든 결과 카드는 다음 4요소를 공통으로 갖습니다.

요소	설명
결과 큰 글씨	사용자가 가장 먼저 봐야 할 핵심 결과
확신 정도	% 또는 색상 배치
출처 / 시간	공식 DB, 최근 업데이트 시각 등 작은 라벨
다음 행동	저장, 고치기, 더 보기 같은 한 줄 행동

3.1 음식 비전 파트의 역할

음식 비전 파트의 목표는 AI가 음식 사진을 완벽하게 판정하는 것이 아닙니다. 1주차의 핵심은 **MVP에서 설명 가능하고 검증 가능한 음식 분석 흐름**을 만드는 것입니다.

사용자가 음식 사진이나 텍스트 식단을 입력하면, 앱은 이를 음식 후보로 만들고 공식 영양 데이터와 연결해야 합니다. 이때 사진만 보고 섭취량까지 단정하면 영양 계산이 크게 흔들릴 수 있으므로, MVP에서는 기본 계산을 100g 기준으로 단순화합니다.

3.2 음식 분석 파이프라인

음식 비전 파트의 전체 흐름은 다음과 같습니다.

```
Input
-> Detection
-> Fusion
-> RDA Match
-> 100g Nutrition Profile
-> Optional Scale
```

단계	역할
Input	음식 사진 또는 텍스트 식단 입력
YOLOv8 mock detector	음식 bbox, class, confidence 후보 생성
Google Vision mock hint adapter	label, object hint, OCR 원문 제공
Fusion engine	YOLO 후보를 중심으로 GCV hint와 OCR을 보강해 최종 후보 정리
RDA matcher	음식 후보를 농진청/RDA 계열 영양 데이터와 연결
Optional Scale	사용자가 g 또는 인분을 입력한 경우 섭취량 기준 재계산

중요한 구조 결정은 GCV를 음식명 확정 엔진으로 쓰지 않는다는 점입니다. GCV label과 OCR은 보조 힌트로 사용하고, 최종 후보는 YOLO primary와 fusion 규칙으로 정리합니다.

3.3 100g 기준 영양 프로필

음식 사진만으로 실제 섭취량을 정확히 추정하는 것은 기술 리스크가 큼니다. 그래서 MVP 기본 경로에서는 이미지 기반 섭취량 추정을 제외하고, 식별된 음식의 **100g 기준 영양 프로필**을 먼저 제공합니다.

결정	이유
사진만으로 섭취량을 단정하지 않음	부정확한 양 추정으로 영양 결과가 왜곡되는 것을 방지
100g 기준 영양정보 제공	사용자에게 음식의 기본 영양 성격을 빠르게 전달
사용자 입력량 기반 재계산	사용자가 g 또는 인분을 입력했을 때만 실제 섭취량 계산
review flag 유지	매칭 실패나 애매한 후보를 확정하지 않고 사용자 검토로 전환

이 방식은 기술적 한계를 숨기는 것이 아니라, 불확실한 추정을 줄이고 사용자가 확인할 수 있는 구조로 만드는 결정입니다.

3.4 응답 구조와 리스크 대응

음식 분석 응답에는 다음 정보가 포함되어야 합니다.

응답 요소	의미
음식 후보	사용자가 선택하거나 수정할 수 있는 후보 목록
100g 기준 영양정보	공식 영양 데이터와 연결된 기본 영양 프로필
review flag	후보가 애매하거나 매칭 실패 시 사용자 검토 필요 표시

대표 리스크와 대응은 다음과 같습니다.

리스크	대응
이미지 기반 섭취량 추정 부정확	100g 기준을 기본으로 두고 사용자 입력량이 있을 때만 재계산
AI Hub 전체 데이터 용량 과다	작은 subset과 mock 기반으로 MVP 흐름 우선 검증
한국 음식 alias와 표기 변형	<code>food_aliases.json</code> 같은 제품 자산으로 관리
의료 표현 오해	진단·치료·처방처럼 보이는 표현을 제외하고 정보 제공형 문구 사용
real GCV/YOLO 전환 시 흔들림	mock과 real 구현이 같은 인터페이스를 따르도록 고정

4.1 알고리즘 설계의 핵심

알고리즘 파트의 핵심은 계산을 더 복잡하게 만드는 것이 아니라, **공식 근거가 있는 계산식과 프로젝트 운영 중 검증해야 할 규칙을 분리**하는 것입니다.

비전공자 관점에서 보면, 이 작업은 AI가 건강 판단을 마음대로 하게 만드는 일이 아닙니다. 근거가 있는 계산은 고정하고, 아직 검증이 필요한 값은 운영 규칙으로 표시해 나중에 검토할 수 있게 만드는 작업입니다.

4.2 공식 기반 계산식과 운영 규칙

구분	항목	설명
공식 기반 계산식	BMI	체형 분류의 기본 기준
공식 기반 계산식	BMR	기초대사량 계산
공식 기반 계산식	HRmax	심박 기반 활동 기준
공식 기반 계산식	KDRIs/DRI	영양 권장 기준
공식 기반 계산식	7-step 체중 예측	BMR, TDEE, 섭취 칼로리, 7,700 kcal/kg 기준 참고 예측
운영 규칙	성별·나이·BMI 보정	사용자 특성별 활동 기준 조정
운영 규칙	질환 가중치	만성질환 맥락을 반영하는 프로젝트 규칙
운영 규칙	부족/낮음/적정/과다 기준	권장량 대비 상태를 단계로 표현
운영 규칙	영양제 매칭 가중치	성분명, 함량, 단위 매칭 기준

공식 기반 계산식은 비교적 안정적인 기준으로 두고, 운영 규칙은 MVP 이후 데이터와 전문가 검토를 통해 조정할 수 있는 영역으로 분리합니다.

4.3 OCR/LLM 결과와 preview 처리

OCR이나 LLM이 만든 영양제 분석 결과는 바로 최종 기록이 아닙니다.

영양제 라벨 촬영

- > OCR 텍스트 추출
- > LLM 구조화
- > Pydantic 검증
- > preview 결과 표시
- > 사용자 수정·승인
- > 최종 기록 저장

이 흐름에서 AI는 판단을 확정하는 주체가 아닙니다. OCR 결과를 정리하고 설명을 돕는 역할이며, 최종 기록은 사용자가 확인한 뒤 저장됩니다.

4.4 안전성과 재현 가능성

사용자에게 보이는 문구는 진단, 치료, 처방처럼 들리지 않게 제한합니다. 표현은 **참고 정보**, **주의 가능성**, **전문가 상담 권장** 형태로 유지합니다.

또한 나중에 같은 입력에 대해 같은 결과가 나왔는지 확인할 수 있도록 다음 정보를 함께 남깁니다.

저장해야 할 정보	이유
입력값 snapshot	어떤 데이터로 계산했는지 확인
결과값 snapshot	사용자에게 어떤 결과를 보여줬는지 확인
알고리즘 버전	계산식 변경 여부 추적
KDRIs 출처 버전	영양 기준 변경 여부 추적
preview 승인 상태	사용자가 확인한 결과인지 구분

알고리즘 파트의 방향은 기능 과장이 아니라 **설명 가능성**, **재현 가능성**, **안전성**을 강화하는 것입니다.

5.1 DB/백엔드 파트의 역할

DB는 단순 저장소가 아니라, Lemon Aid의 로그인, 프로필, 식단, 영양제, 병원성 데이터, Agent 분석 결과를 **user_id 기준으로 연결하는 서비스의 기억 공간**입니다.

사용자는 로그인하고 프로필을 저장하며, 음식·영양제와 건강 데이터를 기록합니다. 백엔드는 이 데이터를 검증하고 필요한 만큼만 Agent에게 전달합니다. Agent 분석 결과는 다시 PostgreSQL에 저장되고, 다음 분석에 필요한 요약은 Agent Memory로 재활용 됩니다.

5.2 전체 데이터 흐름

전체 구조는 다음과 같습니다.

```
Flutter App
-> FastAPI Backend
-> PostgreSQL / Redis
-> 분석 알고리즘 / LLM Agent
```

구성	역할
Flutter App	로그인, 사진 촬영, 프로필 입력, 챗봇 요청
FastAPI Backend	인증, 요청 검증, DB 저장, Agent 호출
PostgreSQL	사용자 정보, 프로필, 음식·영양제 기록, 분석 결과, Agent Memory 장기 저장
Redis	OCR 결과, KDRIs 기준값, API 호출 제한 등 임시 저장
LLM Agent	백엔드가 정리한 최소 데이터로 분석 보조

중요한 점은 Flutter 앱이 DB에 직접 접근하지 않는다는 것입니다. 모든 요청은 FastAPI를 거쳐야 인증, 검증, 저장 정책을 일관되게 적용할 수 있습니다.

5.3 PostgreSQL과 Redis 선택 이유

PostgreSQL은 Lemon Aid 데이터 특성과 잘 맞습니다.

기능	Lemon Aid에서 필요한 이유
JSONB	영양제 성분표, 음식 분석 결과처럼 구조가 유동적인 데이터 저장
TimescaleDB	걸음수, 체중, 심박수처럼 시간 순서로 쌓이는 건강 데이터 처리
RLS	user_id 기준으로 자기 데이터만 접근하도록 제한

Redis는 영속 저장소가 아니라 반복 비용을 줄이기 위한 캐시 계층입니다.

Redis 사용처	목적
OCR 결과 캐시	같은 사진 분석의 반복 호출 비용 절감
KDRIs 기준값 캐시	자주 조회되는 기준값 빠른 반환
rate limit	사용자별 호출 제한 관리

5.4 주요 데이터와 Agent Memory

주요 테이블은 user_id 기준으로 연결됩니다.

데이터	의미
users	로그인 계정 정보
profiles	나이, 성별, 키, 몸무게, 만성질환, 복용 정보
foods / supplements	음식·영양제 마스터 데이터
meals / user_supplements	사용자의 실제 음식·영양제 기록
analysis_results / daily_scores	AI 분석 결과와 식단관리 점수
agent_memory / agent_runs	Agent 요약 기억과 호출 로그

Agent Memory는 모든 원본 데이터를 저장하지 않습니다. 다음 분석에 필요한 최근 검사값 요약, 만성질환 태그, 복용 정보, 최근 주의사항, 식단관리 점수 같은 핵심 요약만 저장합니다.

5.5 보안 방향

Lemon Aid는 민감한 건강 정보를 다루기 때문에 데이터 접근과 저장 범위를 제한해야 합니다.

보안 방향	내용
비밀번호 원문 저장 금지	해시로 검증
JWT 인증	요청마다 사용자 확인
user_id 기준 접근 제한	사용자별 데이터 분리
민감정보 암호화 검토	만성질환, 복약, 검사값 등 보호
감사 로그	민감정보 접근 기록
LLM 전달 데이터 최소화	Agent가 DB 전체를 보지 않도록 제한

DB/백엔드 파트의 핵심은 사용자별 장기 데이터를 안전하게 저장하고, Redis로 반복 비용을 줄이며, LLM Agent에는 필요한 최소 데이터만 전달하는 구조입니다.

Section

6. 1주차 통합 정리

1주차의 핵심은 각 파트가 따로 문서를 만든 것이 아니라, 하나의 서비스 흐름을 나누어 정리했다는 점입니다.

UX는 사용자가 결과를 믿고 고칠 수 있는 화면 규칙을 정리했습니다. 음식 비전은 음식 사진과 텍스트를 분석 후보로 만들고, 100g 기준 영양 프로필과 사용자 입력량 기반 재계산 구조를 잡았습니다. 알고리즘은 공식 기준과 운영 규칙을 분리해 설명 가능성과 안전성을 확보했습니다. DB/백엔드는 사용자별 기록과 Agent 분석 결과를 안전하게 연결하고 재사용하는 구조를 정리했습니다.

이 네 파트는 분리된 작업이 아니라, **음식·영양제 입력이 사용자 건강관리 피드백으로 이어지는 하나의 서비스 흐름**을 구성합니다.